UShell

UShell is a developer-grade runtime console for Unity built around a statically-typed, zero-reflection registration API. It discards the attribute-scanning approach in favour of an explicit Fluent Builder pattern — giving you predictable startup cost, compile-time safety, and full architectural control.

Table of Contents

🏛️ Architecture & Core Philosophy

Before writing a single line of code, it is worth understanding the four principles that govern every design decision in UShell:

1. Zero-Reflection / Explicit Registration. There are no `[ConsoleCommand]` attributes. Commands are constructed using a `ShellBuilder` via a Fluent Builder pattern. Consequence: zero assembly scanning at startup, zero hidden GC spikes, full dead-code-elimination compatibility.

2. The Three-Stage Execution Pipeline.

- **Lexing → Parsing → AST.** User input (e.g., `spawn -count 5 "Orc"`) is tokenized by the `Lexer` and transformed into a typed Abstract Syntax Tree with nodes such as `CommandNode`, `LiteralNode`, `VariableNode`, `ArrayNode`, and `NamedArgumentNode`.
- **Binding.** The `ArgumentBinder` walks the AST and maps each node to a C# object using registered `ITypeParser<T>` implementations.
- **Invocation.** The resolved `ICommandInvoker` (one of the `ActionInvoker` / `FuncInvoker` / `InteractiveFuncInvoker` variants) is called with the fully-typed argument array.

3. Strict Dependency Inversion. `IShellCore` — the engine — knows nothing about `Canvas`, `TextMeshPro`, or `UnityEngine.InputSystem`. It communicates exclusively through two abstractions: `IConsolePrinter` (output) and `IInputProvider` (input). This means the entire core is testable in plain C# without Unity.

4. Interactive, Non-Blocking Execution. Commands may suspend their own execution flow to await user confirmation, collect free-form text input, or stream live progress bars — all without blocking Unity's main thread and without requiring any coroutines.

📦 Installation & Quick Start

Requirements: Unity 2021.3+ · TextMeshPro · Unity New Input System

Option 1: Unity Package (Recommended)

1. Navigate to the **Releases** page.
2. Download the latest `UShell-vX.X.X.unitypackage`.
3. Drag the package into the Unity Project window and import all assets.

Option 2: Package Manager (Git URL)

Window > Package Manager > + > Add package from git URL…

```
https://github.com/Rivgoo/UShell.git?path=/Assets/Plugins/UShell
```

Option 3: Manual

Copy `Assets/Plugins/UShell` from this repository into your project's `Assets` folder.

Prerequisites

- **TextMeshPro:** `Window → TextMeshPro → Import TMP Essential Resources`
- **Input System:** `Project Settings → Player → Active Input Handling` must include the New Input System.

First Run

1. Drag the **UShellManager** prefab from `Assets/Plugins/UShell/` into your scene.
2. Press Play. Press **Backquote / Tilde** to toggle the console.
3. Type `help` and press **Enter**.

⚙ Bootstrapping & Configuration

UShell supports three integration styles. Choose the one that matches your project's architecture.

Option A: Component-Based Setup (Drag & Drop)

`ComponentShellBootstrapper` is a `MonoBehaviour` that lives on the same `GameObject` as `UShellManager`. It reads configuration from the Unity Inspector and discovers `IShellConfigurator` components on child objects automatically.

Inspector fields:

Field	Default	Description
`ActiveEnvironment`	`Development`	The active `EnvironmentTag`. Commands restricted to other environments are excluded at build time.
`MirrorLogsToUnityConsole`	`false`	Also writes every shell log to `UnityEngine.Debug`.
`IncludeConsoleManagementProfile`	`true`	`help`, `clear`, `history`, `alias`, macro commands.
`IncludeApplicationSettingsProfile`	`true`	`screen.resolution`, `time.scale`, `gfx.fps`.
`IncludeSceneManagementProfile`	`true`	`scene.load`, `scene.active`, `scene.list`.
`IncludeMathUtilityProfile`	`true`	`eval`, `random`, `convert`.
`IncludeEnvironmentInfoProfile`	`true`	`info`, `platform`, `game.version`.
`IncludeGameObjectProfile`	`true`	`go.teleport`, `go.active`, `go.info`.
`IncludeRuntimeDiagnosticsProfile`	`true`	`stats`, `mem`, `gc`, `objects`, `layer.find`.
`IncludeHttpProfile`	`true`	`http.get`, `http.post`, `http.put`, `http.delete`.
`IncludeFileIOProfile`	`true`	`file.read`, `file.write`, `screenshot`.

> ⚠ **ARCHITECTURE RULE:** `ComponentShellBootstrapper` auto-discovers every `IShellConfigurator` on its **child** GameObjects. Your custom configurators must be placed as children of the `UShellManager` GameObject in the scene hierarchy.

Option B: Modular Injection (`IShellConfigurator`)

`IShellConfigurator` is the recommended extension point for adding your own commands without touching the `UShellManager` prefab. Implement it on any `MonoBehaviour` and attach it as a child of the `UShellManager` GameObject.

```
using UnityEngine;
using UShell.Runtime.Unity.Bootstrapping;
using UShell.Runtime.Core.Bootstrapping;

public class GameShellConfigurator : MonoBehaviour, IShellConfigurator
{
    [SerializeField] private ItemDatabase _itemDatabase;

    public void Configure(ShellBuilder builder, ShellBootstrapContext context)
    {
        // context gives you access to the shared printer, session state,
history, etc.
        builder.AddProfile(new ItemProfile(context.Printer, _itemDatabase));
        builder.AddProfile(new NetworkAdminProfile(context.Printer));
        builder.AddTypeParser(new ItemIdParser(_itemDatabase));
    }
}
```

`ShellBootstrapContext` properties:

Property	Type	Description
`Printer`	`IConsolePrinter`	Route-safe printer for output from profiles.
`RegistryProxy`	`ICommandRegistry`	A deferred proxy to the final registry — safe to pass to profiles that need to enumerate commands.
`History`	`ICommandHistory`	The shared input history instance.
`SessionState`	`ISessionState`	The shared macro/variable store.
`Controller`	`IShellController`	Allows profiles to request `clear` / `close` lifecycle actions.
`ActiveEnvironment`	`EnvironmentTag`	The environment flag the shell was booted with.

Option C: Pure Code Setup (Zenject / VContainer)

Use `CoreShellBootstrapper` when you want to wire UShell entirely in code — ideal for DI container installers.

```
using UShell.Runtime.Core.Commands;
using UShell.Runtime.Unity.Bootstrapping;
```

```
public class TerminalInstaller : MonoInstaller
{
    [SerializeField] private UShellManager _manager;

    public override void InstallBindings()
    {
        // 1. Instantiate the bootstrapper for the current environment
        var bootstrapper = new CoreShellBootstrapper(
            EnvironmentTag.Development,
            mirrorLogsToUnityConsole: true);

        // 2. Register custom type parsers before profiles
        bootstrapper.AddTypeParser(new PlayerIdParser());

        // 3. Register profiles via a factory delegate — context is injected by
the bootstrapper
        bootstrapper.AddProfile(context =>
            new NetworkAdminProfile(context.Printer,
Container.Resolve<INetworkService>()));

        bootstrapper.AddProfile(context =>
            new PlayerCheatsProfile(context.Printer,
Container.Resolve<IPlayerService>()));

        // 4. Build the core and hand the result to the manager
        BootstrapResult result = bootstrapper.Build();
        _manager.Initialize(result);
    }
}
```

`CoreShellBootstrapper` API:

Method	**Description**
`AddProfile(IShellProfile)`	Registers a pre-constructed profile instance.
`AddProfile(Func<ShellBootstrapContext, IShellProfile>)`	Registers a factory that receives the bootstrap context. Preferred for profiles with DI dependencies.
`AddTypeParser<T>(ITypeParser<T>)`	Registers a custom type parser globally.
`AddConfigurator(IShellConfigurator)`	Attaches a full `IShellConfigurator` instance (advanced).
`Build()`	Wires all dependencies and returns a `BootstrapResult`.

> ⚠ **ARCHITECTURE RULE:** Type parsers must be registered **before** profiles that use them. Registration order matters: parsers first, then profiles.

□ Dynamic Runtime Reconfiguration

UShell supports dynamic runtime modification of the console configuration. Rather than completely tearing down and rebuilding the shell core (which would wipe command history, active session variables, and UI log layouts), UShell utilizes **Configuration Transactions** to mutate the command and parser registries *in-place*.

```
[User API Request]
        │
        ▼
1. Stage Changes (IConfigurationTransaction)
        │
        ▼
2. Concurrency Safety: Wait for all running commands to finish
        │
        ▼
3. Lock System & Close UI (StartSession Lock)
        │
        ▼
4. Perform In-Place Mutations (Command/Parser Registries Updated)
        │
        ▼
5. Unlock System & Complete Transaction
```

This ensures full system stability while allowing you to inject or revoke developer tools as the game context changes (e.g., loading DLCs, entering specific gameplay zones, or changing network states).

The Configuration Transaction Pattern

Dynamic reconfigurations are structured as transactions using the **Unit of Work** pattern. This guarantees that all changes are pre-validated and applied atomically. If any command name or alias collision occurs during staging, the transaction throws an exception and rolls back without corrupting the active console state.

```
// 1. Begin the configuration transaction
IConfigurationTransaction transaction = UShellManager.API.BeginConfiguration();

// 2. Stage multiple operations fluently
transaction
    .AddProfile(new DungeonCheatProfile(UShellManager.API))
    .AddTypeParser(new EnemyTargetParser())
    .RemoveProfile<MainMenuProfile>(); // Remove existing profile by its
compile-time Type

// 3. Apply the changes atomically
await transaction.ApplyAsync();
```

Concurrency & Execution Safety Guarantees

Mutating command dictionaries while the user is executing a command would result in severe runtime exceptions. UShell prevents this using a multi-step safety routine:

1. **Spin-Lock on Active Commands:** Calling `ApplyAsync` executes a non-blocking spin-lock that checks `IInteractiveSession.IsBusy`. It yields execution back to the engine until all currently running asynchronous, interactive, or multi-step commands are fully completed.
2. **System Lock and UI Hiding:** Once active tasks finish, the transaction calls `IShellController.RequestClose()` to hide the console UI. It then calls `IInteractiveSession.StartSession()` to acquire an exclusive system-level lock. This blocks any new command execution and prevents the user from typing.

3. **In-Place Mutation:** The staged additions and removals are processed synchronously within the registers.
4. **Unlocking:** The transaction releases the interactive session lock, returning the console to its standard operating mode.

Transactional Fluent API Reference

The `IConfigurationTransaction` interface provides the following chainable methods:

Method	Description
`AddProfile(IShellProfile profile)`	Stages a profile instance to be added to the console.
`AddTypeParser<T>(ITypeParser<T> parser)`	Stages a custom type parser to be registered.
`RemoveProfile<TProfile>()`	Stages all active profiles of the specified type `TProfile` to be removed.
`RemoveProfile(Type profileType)`	Stages all active profiles of the specified `Type` to be removed.
`RemoveProfile(IShellProfile profile)`	Stages a specific, exact profile instance to be removed.
`RemoveTypeParser<TTarget>()`	Stages the parser mapped to type `TTarget` to be unregistered.
`RemoveTypeParser(Type targetType)`	Stages the parser mapped to `targetType` to be unregistered.
`ApplyAsync(CancellationToken token)`	Executes the safety checks, locks the shell, applies all staged mutations, and unlocks.

Complete Reconfiguration Example

Here is a practical Unity controller managing state transitions when a player enters a specific game zone:

```
using System;
using System.Threading.Tasks;
using UnityEngine;
using UShell.Runtime.Unity.API;
using UShell.Runtime.Core.Configuration;

public class ZoneManager : MonoBehaviour
{
    private IShellProfile? _activeZoneProfile;

    // Called when the player enters a dungeon area
    public async void OnPlayerEnteredDungeon()
    {
        Debug.Log("Loading Dungeon Cheat System...");
```

```
        _activeZoneProfile = new DungeonCheatProfile(UShellManager.API);

        try
        {
            // Begin transactional update
            IConfigurationTransaction transaction =
UShellManager.API.BeginConfiguration();

            transaction
                .AddProfile(_activeZoneProfile)
                .AddTypeParser(new EnemyConfigParser())
                .RemoveProfile<MainMenuProfile>(); // Safely remove using
generic Type argument

            // Waits for running commands, closes UI, locks the terminal,
mutates, and unlocks
            await transaction.ApplyAsync();

            Debug.Log("Console updated successfully for Dungeon mode.");
        }
        catch (Exception ex)
        {
            Debug.LogError($"Failed to reconfigure console: {ex.Message}");
        }
    }

    // Called when the player exits the dungeon back to main menu
    public async void OnPlayerExitedDungeon()
    {
        Debug.Log("Restoring Main Menu Console System...");

        if (_activeZoneProfile == null) return;

        try
        {
            IConfigurationTransaction transaction =
UShellManager.API.BeginConfiguration();

            transaction
                .RemoveProfile(_activeZoneProfile)               // Remove by
concrete instance
                .RemoveProfile(typeof(DungeonCheatProfile))        //
(Alternative) Remove by Type object
                .RemoveTypeParser<EnemyConfig>()                  // Unregister
custom parser
                .AddProfile(new MainMenuProfile(UShellManager.API));

            await transaction.ApplyAsync();
        }
        catch (Exception ex)
        {
            Debug.LogError($"Failed to restore console: {ex.Message}");
        }
    }
}
```

🛠 Profiles & the Fluent API

A **Profile** is a cohesive grouping of related commands — analogous to a controller in MVC. Each profile is responsible for one concern: `AudioProfile`, `NetworkAdminProfile`,

`CheatProfile`, etc.

Creating a Profile

Inherit from `ShellProfile` (not `IShellProfile` directly). The base class provides a protected `Printer` field and a suite of output helper methods. Implement the `Configure(ICommandBuilder builder)` abstract method.

```
using UShell.Runtime.Core.Abstractions;
using UShell.Runtime.Core.Commands.Fluent;
using UShell.Runtime.Core.Output;

public class AudioProfile : ShellProfile
{
    private readonly IAudioService _audio;

    public AudioProfile(IConsolePrinter printer, IAudioService audio) :
base(printer)
    {
        _audio = audio;
    }

    protected override void Configure(ICommandBuilder builder)
    {
        builder.WithName("audio.volume")
            .WithDescription("Gets or sets the master volume (0.0 – 1.0).")
            .AddOptionalParameter<float>("value", -1f)
            .Executes<float>(SetOrGetVolume);

        builder.WithName("audio.mute")
            .WithDescription("Toggles the master mute state.")
            .Executes(ToggleMute);
    }

    private void SetOrGetVolume(float value)
    {
        if (value < 0f)
        {
            PrintSuccess($"Master volume: {_audio.MasterVolume:F2}");
            return;
        }
        _audio.MasterVolume = Mathf.Clamp01(value);
        PrintSuccess($"Master volume set to {_audio.MasterVolume:F2}.");
    }

    private void ToggleMute()
    {
        _audio.IsMuted = !_audio.IsMuted;
        PrintWarning(_audio.IsMuted ? "Audio muted." : "Audio unmuted.");
    }
}
```

> ⚠ **ARCHITECTURE RULE:** `ShellProfile` must not reference any `UnityEngine.UI` or scene-graph types directly. The profile's only output channel is `Printer`. Inject all game-service dependencies through the constructor.

`ICommandBuilder`

The entry point for every command declaration. Accessed as the `builder` parameter inside `Configure`.

Method	Returns	Description
`WithName(string name)`	`ICommandConfigurator`	Starts a new command chain. Names are case-insensitive, no spaces/quotes/brackets/commas allowed.

`ICommandConfigurator` — Metadata

Method	Returns	Description
`.WithDescription(string desc)`	`ICommandConfigurator`	Sets the `help` summary.
`.WithAlias(string alias)`	`ICommandConfigurator`	Adds an alternative trigger name. Chainable — call multiple times for multiple aliases.
`.RestrictedTo(EnvironmentTag tags)`	`ICommandConfigurator`	Bitmask filter. Commands outside the active environment are silently excluded at bootstrap. `EnvironmentTag.Any` (default) appears in all environments.

`EnvironmentTag` values:

Value	Bitmask	Intended use
`Editor`	`1`	Unity Editor only
`Development`	`2`	Development builds
`Release`	`4`	Production builds
`Any`	`7`	Always available (default)

```
// Strip cheat commands from Release builds automatically
builder.WithName("give.item")
    .RestrictedTo(EnvironmentTag.Editor | EnvironmentTag.Development)
    ...
```

`ICommandConfigurator` — Parameter Binding

> ⚠ **ARCHITECTURE RULE:** Required parameters **must** be declared before optional parameters. Violating this order throws a `ShellConfigurationException` at startup. Positional argument binding follows declaration order exactly.

Method	Description
`.AddParameter<T>(string name)`	Declares a **required** positional parameter of type `T`. `name` is used for named argument syntax (e.g., `-count 5`). `T` must have a registered `ITypeParser<T>`.
`.AddOptionalParameter<T>(string name, T defaultValue)`	Declares an **optional** parameter. If the user omits it, `defaultValue` is injected silently.
`.WithSuggestions(IEnumerable<string> list)`	Attaches a static autocomplete list to the **last declared parameter**.
`.WithSuggestions(Func<SuggestionContext, IEnumerable<string>> provider)`	Attaches a dynamic suggestion provider to the **last declared parameter**.
`.WithSuggestions(ISuggestionProvider provider)`	Attaches a reusable `ISuggestionProvider` class to the **last declared parameter**.
`.WithTimeout(TimeSpan timeout)`	**Required** before `.ExecutesInteractiveAsync(...)`. Sets the maximum wall-clock duration before the interactive session is force-cancelled.

Supported parameter types out of the box:

Category	Types
Primitives	`int`, `uint`, `long`, `ulong`, `short`, `ushort`, `byte`, `sbyte`, `float`, `double`, `decimal`, `bool`, `char`, `string`
System	`Guid`, `DateTime`, `TimeSpan`
Unity structs	`Vector2`, `Vector3`, `Vector4`, `Vector2Int`, `Vector3Int`, `Quaternion`, `Color`, `Rect`, `Bounds`
Unity objects	`GameObject` (resolved via `GameObject.Find`), `Transform`
Special	Any `enum` (case-insensitive, suggestions auto-generated), `T[]`, `List<T>`, `HashSet<T>`, `Stack<T>`, `Queue<T>`

Argument syntax examples in the console:

```
# Positional
> spawn Orc 3
```

```
# Named (any order)
> spawn -name Orc -count 3

# Array literal
> batch.exec [kill, respawn, heal]

# Session variable
> $boss = spawn Dragon
> set_health $boss 9999
```

`ICommandConfigurator` — Execution Delegates

Every configuration chain **must** terminate with exactly one `Executes*` call. The type parameters of the delegate **must** match the parameter types declared via `AddParameter` / `AddOptionalParameter` in the same order.

Overload family	Signature pattern	Notes
`Executes(Action)`	No parameters	
`Executes<T1…T5>(Action<T1…T5>)`	Up to 5 typed params	Synchronous, no return value
`ExecutesReturning<TResult>(Func<TResult>)`	No params, has return	Return value is printed as a success log
`ExecutesReturning<T1…T5, TResult>(Func<T1…T5, TResult>)`	Up to 5 params + return	Return value is printed as a success log
`ExecutesAsync(Func<Task>)`	No params, async	Exception is caught and printed gracefully
`ExecutesAsync<T1…T5>(Func<T1…T5, Task>)`	Up to 5 params, async	
`ExecutesInteractiveAsync(Func<ICommandContext, Task>)`	Context only	Locks the shell. Requires `.WithTimeout()`.
`ExecutesInteractiveAsync<T1…T5>(Func<ICommandContext, T1…T5, Task>)`	Context + up to 5 params	`ICommandContext` is always the first argument.

> ⚠ **ARCHITECTURE RULE:** If the count or types of the generic arguments on `Executes*` do not match the declared parameters, a `ShellConfigurationException` is thrown immediately at startup — not at runtime when a user types the command.

Complete Profile Example

```
using System;
using System.Collections.Generic;
using UShell.Runtime.Core.Abstractions;
using UShell.Runtime.Core.Commands;
using UShell.Runtime.Core.Commands.Fluent;
using UShell.Runtime.Core.Output;
using UShell.Runtime.Core.Suggestions;

public class ItemProfile : ShellProfile
{
    private readonly IItemDatabase _db;

    public ItemProfile(IConsolePrinter printer, IItemDatabase db) :
base(printer)
    {
        _db = db;
    }

    protected override void Configure(ICommandBuilder builder)
    {
        builder.WithName("item.give")
            .WithDescription("Adds an item to the player's inventory.")
            .WithAlias("give")
            .RestrictedTo(EnvironmentTag.Editor | EnvironmentTag.Development)
            .AddParameter<string>("itemId")
                .WithSuggestions(GetItemSuggestions)          // dynamic
            .AddOptionalParameter<int>("amount", 1)
            .AddOptionalParameter<bool>("equip", false)
            .Executes<string, int, bool>(GiveItem);

        builder.WithName("item.list")
            .WithDescription("Prints all items in the player's inventory.")
            .Executes(ListItems);
    }

    private void GiveItem(string id, int amount, bool equip)
    {
        if (!_db.TryGet(id, out var item))
        {
            PrintError($"Unknown item id '{id}'.");
            return;
        }
        Inventory.Local.Add(item, amount, equip);
        PrintSuccess($"Gave {amount}× {item.DisplayName}. Equipped: {equip}");
    }

    private void ListItems()
    {
        var items = Inventory.Local.GetAll();
        var headers = new[] { "ID", "Name", "Qty" };
        var rows = new List<IReadOnlyList<string>>();
        foreach (var entry in items)
            rows.Add(new[] { entry.Id, entry.DisplayName, entry.Count.ToString()
});
        PrintTable(headers, rows);
    }

    private IEnumerable<string> GetItemSuggestions(SuggestionContext ctx)
        => _db.GetAllIds();
}
```

Console usage:

```
> item.give sword 2 true
  Gave 2× Iron Sword. Equipped: True

> give potion 5
  Gave 5× Health Potion. Equipped: False

> item.list
  ID        | Name           | Qty
  ----------+----------------+----
  sword     | Iron Sword     | 2
  potion    | Health Potion  | 5
```

⏳ Interactive Commands (`ICommandContext`)

Interactive commands can suspend their execution to collect input, stream live diagnostics, or confirm destructive actions — all without blocking the main thread.

Three hard rules for interactive commands:

1. Call `.WithTimeout(TimeSpan)` before `.ExecutesInteractiveAsync(...)` — without it, a `ShellConfigurationException` is thrown at startup.
2. The first argument of the delegate is always `ICommandContext`. Do not declare it as a parameter via `AddParameter`.
3. Always respect `ctx.Token` — pass it to all awaitable calls. Pressing `Escape` or hitting the timeout triggers cancellation.

`ICommandContext` API

Member	Type	Description
`Token`	`CancellationToken`	Triggered on timeout or user Escape. Pass to all awaitable calls.
`ConfirmAsync(string msg)`	`Task<bool>`	Suspends execution. Displays `msg (y/n)` and awaits user input. Returns `true` for `y`/`yes`.
`PromptAsync(string msg)`	`Task<string>`	Suspends execution. Displays `msg` and awaits a free-form string.
`CreateProgressBar(string taskName)`	`IProgressReporter`	Spawns a live progress bar in the log stream. Dispose it to mark 100% complete.
`Print(string)`	`void`	Standard log. Thread-safe during async execution.
`PrintSuccess(string)`	`void`	Success (green) log.
`PrintWarning(string)`	`void`	Warning (amber) log.

Member	Type	Description
`PrintError(string)`	`void`	Error (red) log.

`IProgressReporter` API

Member	Description
`Report(float progress, string status)`	Updates the bar. `progress` is `0.0–1.0`. Disposing the reporter completes the bar automatically.

Interactive Example

```
builder.WithName("db.rebuild")
    .WithDescription("Wipes and rebuilds the local player database.")
    .WithTimeout(TimeSpan.FromMinutes(2))
    .ExecutesInteractiveAsync(RebuildDatabaseAsync);

private async Task RebuildDatabaseAsync(ICommandContext ctx)
{
    bool confirmed = await ctx.ConfirmAsync("This will erase all local data.
Proceed?");
    if (!confirmed)
    {
        ctx.PrintWarning("Operation cancelled.");
        return;
    }

    string env = await ctx.PromptAsync("Target environment
(dev/staging/prod):");
    ctx.PrintWarning($"Targeting: {env}");

    using (IProgressReporter bar = ctx.CreateProgressBar("Rebuilding database"))
    {
        for (int step = 0; step <= 10; step++)
        {
            ctx.Token.ThrowIfCancellationRequested();
            bar.Report(step / 10f, $"Step {step}/10");
            await Task.Delay(500, ctx.Token);
        }
    }

    ctx.PrintSuccess("Database rebuilt successfully.");
}
```

Console interaction:

```
> db.rebuild
  This will erase all local data. Proceed? (y/n)
> y
  Target environment (dev/staging/prod):
> staging
  ⚠ Targeting: staging
  [Rebuilding database] ████████████████████ 100%
  ✓ Database rebuilt successfully.
```

⚠ **ARCHITECTURE RULE:** When an interactive command is running, the shell is

> **locked**. The user cannot type or execute other commands. The only allowed input is text for pending `PromptAsync` / `ConfirmAsync` calls. Pressing `Escape` calls `IInteractiveSession.Cancel()`, which triggers the `CancellationToken`.

🔍 Autocomplete & Suggestions

UShell uses a **Fuzzy Matcher** backed by a Trie for O(k) prefix lookup and a scoring heuristic for ranking results. Tab-completion fills in the top-scored suggestion. Up to 5 ranked signatures are displayed below the input field as ghost tooltips.

Static Suggestions

Provide a fixed list attached to the last declared parameter:

```
.AddParameter<string>("biome")
.WithSuggestions(new[] { "forest", "desert", "tundra", "ocean" })
```

Dynamic Suggestions

Compute the list at suggestion-request time. Receives a `SuggestionContext` with the partial text already typed:

```
.AddParameter<string>("playerName")
.WithSuggestions(ctx =>
    ServerState.GetConnectedPlayers()
               .Where(p => p.Name.StartsWith(ctx.PartialValue,
StringComparison.OrdinalIgnoreCase))
               .Select(p => p.Name))
```

Reusable Provider Class

For complex logic shared across multiple commands, implement `ISuggestionProvider`:

```
public class SceneNameProvider : ISuggestionProvider
{
    public IEnumerable<string> GetSuggestions(SuggestionContext context)
    {
        int count = SceneManager.sceneCountInBuildSettings;
        for (int i = 0; i < count; i++)
        {
            string path = SceneUtility.GetScenePathByBuildIndex(i);
            string name = System.IO.Path.GetFileNameWithoutExtension(path);
            if (name.StartsWith(context.PartialValue,
StringComparison.OrdinalIgnoreCase))
                yield return name;
        }
    }
}

// Registration:
.AddParameter<string>("sceneName")
.WithSuggestions(new SceneNameProvider())
```

> **Note:** `enum` and `bool` parameters automatically generate their own suggestions (`Enum.GetNames` and `["true","false","1","0"]` respectively). You do not

need to configure them manually.

□ Custom Type Parsers (`ITypeParser<T>`)

The `ArgumentBinder` converts raw string tokens into C# types. To teach UShell how to interpret a custom type, inherit from `TypeParser<T>` and implement `ParseTyped`.

Implementation Contract

```
using UShell.Runtime.Core.Execution;
using UShell.Runtime.Core.Diagnostics;
using UShell.Runtime.Core.Parsing.Types;

// Custom type: public readonly struct PlayerId { public int Value; }

public class PlayerIdParser : TypeParser<PlayerId>
{
    private readonly IPlayerRegistry _registry;

    public PlayerIdParser(IPlayerRegistry registry)
    {
        _registry = registry;
    }

    public override ExecutionResult<PlayerId> ParseTyped(string input)
    {
        if (!int.TryParse(input, out int id))
        {
            return ExecutionResult<PlayerId>.Failure(
                ShellError.Create(ShellErrorCode.Bind_CustomError, -1,
                    $"'{input}' is not a valid integer."));
        }

        if (!_registry.Exists(id))
        {
            return ExecutionResult<PlayerId>.Failure(
                ShellError.Create(ShellErrorCode.Bind_CustomError, -1,
                    $"No player with ID {id} found."));
        }

        return ExecutionResult<PlayerId>.Success(new PlayerId { Value = id });
    }
}
```

Use `ShellError.Create(ShellErrorCode.Bind_CustomError, position, message)` for failures. The `position` argument is used by the `ErrorVisualizer` to point a caret at the offending token in the error output.

Registration

In your `IShellConfigurator.Configure` or `CoreShellBootstrapper`:

```
builder.AddTypeParser(new PlayerIdParser(playerRegistry));
```

Now `AddParameter<PlayerId>("target")` works in any profile.

ScriptableObject Lookup Example

```
public class ItemDefinitionParser : TypeParser<ItemDefinition>
{
    private readonly ItemDatabase _db;

    public ItemDefinitionParser(ItemDatabase db) { _db = db; }

    public override ExecutionResult<ItemDefinition> ParseTyped(string input)
    {
        var item = _db.FindById(input);
        if (item == null)
            return ExecutionResult<ItemDefinition>.Failure(
                ShellError.Create(ShellErrorCode.Bind_CustomError, -1,
                    $"Item '{input}' not found in database."));

        return ExecutionResult<ItemDefinition>.Success(item);
    }
}
```

□ Session State & Macros

`ISessionState` is a runtime key-value store that survives across command calls within the same play session.

Variable Syntax in the Console

```
# Assign a literal
> $speed = 5.5

# Assign the return value of a command (ExecutesReturning)
> $boss = spawn Dragon

# Use variables as arguments
> set_speed $boss $speed

# Math eval with variables
> eval $speed * 2 + 10
  21.0
```

Built-In Macro Commands

Command	Description
`macro.list`	Prints all currently stored variables and their values.
`macro.delete <name>`	Deletes a specific variable from the session.
`macro.clear`	Wipes all variables.

Accessing Session State from C#

`ISessionState` is available in `ShellBootstrapContext.SessionState` during bootstrapping and can be injected into profiles that need to read or write variables programmatically:

Method	Description
`TryGetValue(string name, out object value)`	Returns `true` and sets `value` if the variable exists.
`SetValue(string name, object value)`	Programmatically writes a variable.
`Clear()`	Removes all stored variables.

□ Output, Formatting & Theming

`ShellProfile` Output Methods

Use these instead of `Debug.Log` inside any `ShellProfile`. They route through `IConsolePrinter`, ensuring theme-aware colour application and correct mirroring to the Unity Console if enabled.

Method	Log colour	Use for
`Print(string)`	Default (warm grey)	Neutral informational text
`PrintSuccess(string)`	Green	Confirmations, results
`PrintWarning(string)`	Amber	Non-fatal issues, user hints
`PrintError(string)`	Red	Failures, validation errors
`PrintList(string title, IReadOnlyList<string> items, int limit)`	Mixed	Numbered lists with truncation
`PrintTable(headers, rows, TableStyle)`	Mixed	ASCII grid tables

`PrintTable` and `TableStyle`

```
var headers = new[] { "Command", "Alias", "Description" };
var rows = new List<IReadOnlyList<string>>
{
    new[] { "scene.load", "sl",  "Loads a scene by name or index." },
    new[] { "scene.list", "",    "Lists all scenes in the build." },
};

// TableStyle.Standard — separator only under header row
PrintTable(headers, rows, TableStyle.Standard);

// TableStyle.Grid — separator between every row
PrintTable(headers, rows, TableStyle.Grid);
```

`ShellPalette` — Design Tokens

Never hardcode hex colours in profile output. `ShellPalette` provides semantic constants that

adapt if the theme changes.

Category	Constants
Text	`TextPrimary`, `TextSecondary`, `TextMuted`, `TextDim`, `TextHint`
Semantic	`Success`, `Warning`, `Error`, `ErrorMuted`
Syntax highlighting	`SyntaxCommand`, `SyntaxParam`, `SyntaxType`, `SyntaxValue`, `SyntaxNumber`, `SyntaxKeyword`, `SyntaxAlias`, `SyntaxUsage`
Decoration	`HeaderRule`, `TableHeader`, `TableSeparator`, `TableCell`, `Ruler`
Environment badges	`BadgeEditor`, `BadgeDev`, `BadgeRelease`
Metrics	`StatGood`, `StatWarn`, `StatCritical`, `StatLabel`, `StatUnit`

`ShellPalette` also exposes two helpers:

```
// Returns StatGood / StatWarn / StatCritical — lower is worse (e.g., frame
time)
string color = ShellPalette.MetricColor(frameTimeMs, warnThreshold: 16f,
critThreshold: 33f);

// Higher is worse inverted — lower is worse (e.g., FPS)
string color = ShellPalette.MetricColorInverted(fps, goodThreshold: 60f,
warnThreshold: 30f);
```

`RichText` Utility

`RichText` wraps TextMeshPro rich text tags with a concise API:

```
using UShell.Runtime.Core.Output.Formatting;

string colored  = RichText.Color("text", ShellPalette.SyntaxValue);
string bold     = RichText.Bold("important");
string italic   = RichText.Italic("note");
string combined = RichText.Bold(RichText.Color("5000 HP",
ShellPalette.StatCritical));

Print($"Spawned {RichText.Color("Dragon", ShellPalette.SyntaxValue)} with
{combined}.");
```

`ProfileFormatter` Layout Helpers

```
using UShell.Runtime.Core.Output.Formatting;

// Produces: ── my section ──────────────────
string header = ProfileFormatter.FormatSectionHeader("my section");

// Appends "  Key    : Value\n" with alignment padding
var sb = new StringBuilder();
ProfileFormatter.AppendKeyValue(sb, "Scene", "Level_01");
ProfileFormatter.AppendKeyValue(sb, "Build Index", "3");
```

```
Print(sb.ToString());
```

> ⚠ **ARCHITECTURE RULE:** Never hardcode hex colour literals directly in `Print` calls. Always use `ShellPalette` constants. This is the only way to guarantee that your custom profile output remains visually coherent if the theme asset is swapped.

🎛 UI Customization & Input Adapters

`UShellUIConfiguration` ScriptableObject

Located at `Assets/Plugins/UShell/UShell_UI_Configuration.asset`. Assign a modified copy to the `ConsoleView` component or create a new one via `Assets → Create → UShell → UI Configuration`.

Field	Description
`MaxLogs`	Maximum number of log entries kept alive in the scroll view before oldest are destroyed.
`MainFont`	`TMP_FontAsset` used across all console elements.
`InputFontSize`	Font size for the command input field.
`LogFontSize`	Font size for all log entry lines.
`ForceGlobalMonospace`	Forces all TextMeshPro text to use a fixed character width. **Enable this if your font is not monospaced**, otherwise ASCII table columns will misalign.
`GlobalMonospaceWidth`	Character width in `em` units when `ForceGlobalMonospace` is on. `0.6` works for most proportional fonts.
`GhostTextColor`	Colour of the faded autocomplete overlay text that appears over the user's input.
`PromptColor`	Colour of the `>` prompt prefix.
`SuggestionBackgroundColor`	Background tint for the command signature tooltip panel.
`StandardLogColor`	Default text colour.
`SuccessLogColor`	Green text colour.
`WarningLogColor`	Amber text colour.
`ErrorLogColor`	Red text colour.
`StandardIcon` / `SuccessIcon` / `WarningIcon` / `ErrorIcon`	Per-severity icon sprites rendered next to each log line.

Custom Input Provider (`IInputProvider`)

UShell ships with `InputSystemProvider` (Unity New Input System). To replace it — for Rewired, legacy `Input`, or any other system — implement `IInputProvider` on a `MonoBehaviour` and attach it to the `UShellManager` GameObject. `ConsoleRuntimeEngine` will detect and use it automatically.

```
using System;
using UnityEngine;
using UShell.Runtime.Unity.Inputs;

public class RewiredInputProvider : MonoBehaviour, IInputProvider
{
    public event Action OnToggleConsole;
    public event Action OnSubmit;
    public event Action OnHistoryUp;
    public event Action OnHistoryDown;
    public event Action OnAutocomplete;
    public event Action OnEscape;

    private bool _uiActive;

    private void Update()
    {
        // Replace with your Rewired player.GetButtonDown(...) calls
        if (Input.GetKeyDown(KeyCode.BackQuote))  OnToggleConsole?.Invoke();
        if (!_uiActive) return;
        if (Input.GetKeyDown(KeyCode.Return))     OnSubmit?.Invoke();
        if (Input.GetKeyDown(KeyCode.UpArrow))    OnHistoryUp?.Invoke();
        if (Input.GetKeyDown(KeyCode.DownArrow))  OnHistoryDown?.Invoke();
        if (Input.GetKeyDown(KeyCode.Tab))        OnAutocomplete?.Invoke();
        if (Input.GetKeyDown(KeyCode.Escape))     OnEscape?.Invoke();
    }

    public void SetUIInputActive(bool active) => _uiActive = active;
}
```

Default key bindings (`InputSystemProvider`):

Action	**Key**
Toggle console	Backquote / Tilde
Submit command	Enter / Numpad Enter
History up	Up Arrow
History down	Down Arrow
Autocomplete	Tab
Abort / Close	Escape

□ Programmatic API (`IUShellAPI`)

`UShellManager.API` is a globally accessible, statically typed facade that lets you interact with the console programmatically from any game system — without direct coupling to internal shell

dependencies or low-level UI implementations.

Quick Examples

```
using UnityEngine;
using UShell.Runtime.Unity.API;
using UShell.Runtime.Core.Configuration;

public class GameManager : MonoBehaviour
{
    private void OnEnable()
    {
        // React to visibility changes (e.g., disabling player controls)
        UShellManager.API.OnConsoleOpened += LockPlayerControls;
        UShellManager.API.OnConsoleClosed += UnlockPlayerControls;

        // Log system messages directly to the terminal from external systems
        UShellManager.API.OnLogAdded += LogToExternalCloud;
    }

    private void OnDisable()
    {
        UShellManager.API.OnConsoleOpened -= LockPlayerControls;
        UShellManager.API.OnConsoleClosed -= UnlockPlayerControls;
        UShellManager.API.OnLogAdded -= LogToExternalCloud;
    }

    public void TriggerDiagnostics()
    {
        // Force-execute any registered command as if the user typed it
        UShellManager.API.ExecuteCommand("stats");
    }

    public async void LoadDeveloperDLC()
    {
        // Dynamically alter console commands and type parsers on the fly
        IConfigurationTransaction transaction =
UShellManager.API.BeginConfiguration();

        transaction
            .AddProfile(new DebugCheatsProfile(UShellManager.API))
            .AddTypeParser(new SpawnWeightParser())
            .RemoveProfile<ProductionTelemetryProfile>();

        // Transaction waits for executing tasks, locks input, and mutates in-
place
        await transaction.ApplyAsync();
    }

    private void LockPlayerControls() => Debug.Log("Controls Locked.");
    private void UnlockPlayerControls() => Debug.Log("Controls Unlocked.");
    private void LogToExternalCloud(UShell.Runtime.Core.Output.LogEntry entry) {
/* ... */ }
}
```

`IUShellAPI` Reference

Properties

Property	Type	Description
`Core`	`IShellCore`	Provides raw compile-time read-only access to the shell core engine registers (History, Session, ParserRegistry).
`View`	`ConsoleView`	Provides direct access to the console canvas UI view layout controller.
`Printer`	`IConsolePrinter`	Provides direct access to the active output console printer instance.
`Controller`	`IShellController`	Provides direct access to the lifecycle event aggregation controller.
`IsVisible`	`bool`	Returns `true` if the console overlay canvas is currently active on screen.
`IsExecutingInteractiveCommand`	`bool`	Returns `true` if an interactive command currently holds an exclusive session lock.
`CurrentInputText`	`string`	Retrieves the exact raw string currently entered into the terminal's input field.
`TotalLogsCount`	`int`	Returns the current total number of visual log items rendered in the viewport.

Events

Event	Signature	Fires When
`OnConsoleOpened`	`Action`	The console canvas is activated and acquires visual/keyboard focus.
`OnConsoleClosed`	`Action`	The console canvas is deactivated and relinquishes focus.
`OnConsoleCleared`	`Action`	All logged visual entries are wiped via a clear request.
`OnInputTextChanged`	`Action<string>`	The text inside the input field is modified by typing or autocomplete.

Event	Signature	Fires When
`OnCommandExecuting`	`Action<string>`	A command string has been submitted and is about to enter the parse pipeline.
`OnCommandExecuted`	`Action<string, ExecutionResult<object?>>`	A command execution completes, yielding either a result payload or a syntax/binding error.
`OnLogAdded`	`Action<LogEntry>`	A new log entry (Standard, Success, Warning, or Error) is dispatched to the output pipeline.

Methods

Method	Return Type	Description
`Show()`	`void`	Forces the console overlay UI to open, displaying it on screen and focusing the cursor.
`Hide()`	`void`	Closes the console overlay UI and releases focus.
`Clear()`	`void`	Programmatically triggers a complete clearing of all visual log items.
`ExecuteCommand(string rawCommand)`	`void`	Executes a raw command line string programmatically (e.g. `ExecuteCommand("time.scale 0.5")`).
`BeginConfiguration()`	`IConfigurationTransaction`	Spawns a fluent, transactional Unit of Work to dynamically add/remove profiles and type parsers in-place during runtime.

📚 Built-In Profiles Reference

All built-in profiles are toggled via the `ComponentShellBootstrapper` Inspector or explicitly registered in `CoreShellBootstrapper`.

`ConsoleManagementProfile`

Command	Alias	Description
`help`	—	Lists all available commands. `help <command>` shows full

Command	Alias	Description
		signature.
`clear`	`cls`	Clears the console log.
`history`	—	Prints command history. `history -limit N` controls count.
`alias`	—	Lists all registered command aliases.
`macro.list`	—	Prints all active session variables.
`macro.delete <name>`	—	Deletes a specific session variable.
`macro.clear`	—	Clears all session variables.

`EnvironmentInfoProfile`

Command	Description
`info`	Prints Unity version, platform, and device info.
`platform`	Prints current `RuntimePlatform`.
`game.version`	Prints `Application.version`.
`env`	Prints the active `EnvironmentTag`.

`ApplicationSettingsProfile`

Command	Description
`screen.resolution <width> <height>`	Changes screen resolution.
`gfx.fps <limit>`	Sets `Application.targetFrameRate`.
`time.scale <value>`	Sets `Time.timeScale`.

`SceneManagementProfile`

Command	Description
`scene.load <name|index>`	Loads a scene.
`scene.active`	Prints the active scene name and index.
`scene.list`	Lists all scenes in the build.

`MathUtilityProfile`

Command	Description
`eval <expression>`	Full math expression evaluator. Supports `+`, `-`, `*`, `/`, `%`, `^`, `sqrt`, `sin`, `cos`, `tan`, `log`, `abs`, `min`, `max`, `round`, `floor`, `ceil`, `fact`, `hex()`, `bin()`, and more.
`random <min> <max>`	Returns a random float between min and max.
`convert <value> <from> <to>`	Converts between units (metric/imperial, hex/bin/dec).

`RuntimeDiagnosticsProfile`

Command	Description
`stats`	Live FPS and frame time.
`mem`	GC heap size and total reserved memory.
`gc`	Forces `GC.Collect()` and reports freed bytes.
`objects`	Counts all active `UnityEngine.Object` instances by type.
`layer.find <layer>`	Lists all GameObjects on a specified layer.
`prefs.clear`	Calls `PlayerPrefs.DeleteAll()`.

`GameObjectProfile`

Command	Description
`go.teleport <name> <x> <y> <z>`	Teleports a GameObject by name to a world position.
`go.active <name> <state>`	Sets `GameObject.SetActive`.
`go.info <name>`	Prints position, rotation, scale, components.

`HttpProfile`

Interactive HTTP commands that pause the console while awaiting a network response.

Command	Description
`http.get <url>`	Sends a GET request and prints the response body.
`http.post <url> <body>`	Sends a POST request with a JSON body.
`http.put <url> <body>`	Sends a PUT request.
`http.delete <url>`	Sends a DELETE request.

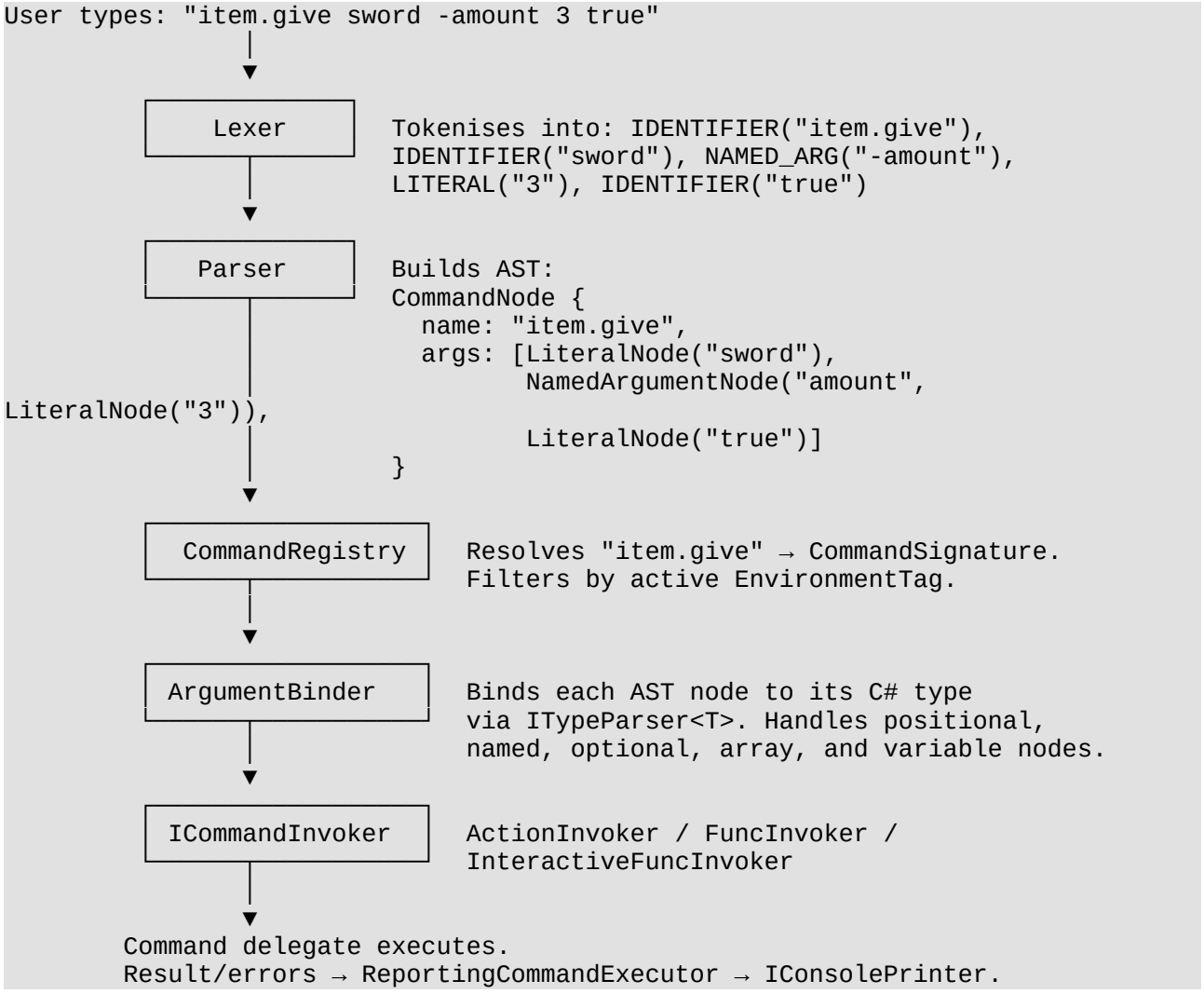
`FileIOProfile`

Command	Description
`screenshot`	Captures a screenshot to `Application.persistentDataPath`.
`file.read <path>`	Reads and prints a text file.
`file.write <path> <content>`	Writes a string to a file.

□ Architecture Deep Dive

This section is for contributors or developers extending the core subsystems.

Execution Pipeline Detail



Key Interfaces and Responsibilities

Interface	Responsibility
`IShellCore`	Aggregates `Registry`, `Executor`, `History`, `InteractiveSession`.
`ICommandRegistry`	Read-only: `TryGetCommand`, `GetSuggestions`, `GetAllCommands`, `GetCompactSignature`.
`ICommandExecutor`	`Execute(string)` + `OnExecuting` / `OnExecuted` events.
`IConsolePrinter`	`Print(LogEntry)`, `UpdatePrint(Guid, LogEntry)`, `PrintTable(...)`. Abstracts the UI completely.
`IInteractiveSession`	Manages the lock state during interactive command execution (`IsBusy`, `IsWaitingForPrompt`, `StartSession`, `EndSession`, `SubmitInput`, `Cancel`).
`IShellController`	Provides `RequestClear()` / `RequestClose()` for profiles that need to affect shell lifecycle without referencing the view.
`IArgumentBinder`	Maps `IReadOnlyList<SyntaxNode>` to `object[]` using registered parsers.
`ITypeParser`	Non-generic base. `TypeParser<T>` provides the `Parse(string) → ExecutionResult<object?>` bridge.
`ITypeParser<T>`	`ParseTyped(string) → ExecutionResult<T>`. Implement this for custom types.
`ISuggestionProvider`	`GetSuggestions(SuggestionContext) → IEnumerable<string>`.
`IInputProvider`	Hardware abstraction: six events + `SetUIInputActive(bool)`.
`IShellProfile`	`RegisterCommands(ICommandBuilder builder)`. Implemented by `ShellProfile`.
`IShellConfigurator`	`Configure(ShellBuilder builder, ShellBootstrapContext context)`. Unity-layer extension point.
`IUShellAPI`	Public facade on `UShellManager.API`.

Assembly Layout

Assembly	`noEngineReferences`	Contains
`UShell.Runtime.Core`	`true`	All engine-agnostic logic: parsing, binding, execution, history, session, output abstractions, formatting. Fully unit-testable without Unity.
`UShell.Runtime.Unity`	`false`	Unity-specific: bootstrappers, `MonoBehaviour` components, Unity type parsers, `UnityConsolePrinter`, `InputSystemProvider`, UI components, built-in profiles.

□ License